



BILDUNG. FREUDE INKLUSIVE.

Web design at IT-L@B

Introduction to HTML and
CSS Flex layouts with CSS

Revised: 03.02.2025

by Dipl.-Ing. (FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

Work order:

1. Read the script **in full**!
2. **Program** examples presented - also in your own variants. Avoid copying other people's program scripts!
3. **Complete** the assignment at the end of the lesson!
4. **Test** the website on different web browsers!

***The code examples are inserted as graphics in this document.
This means: Copy&Paste is not a possible solution!***

Do NOT work with ChatGPT or other AI systems, because YOU are supposed to learn programming!!!!

Calculated processing time

The task is designed in such a way that you should need a maximum of **24 hours to** complete it, including reading this lesson trying out the examples and possibly doing your own research on the Internet. If you have any problems (e.g. difficulties understanding the German language), please contact your trainer.

The content of this module is very extensive. Read the script carefully and work through the exercises so that you can also answer later exam questions.

The following rules apply to the submission of tasks:

Official apprentices of the IT-L@B:

IT-@B apprentices have access to the Moodle learning system. Call up the following course:

20E 55 - User Experience - IT-L@B Villach

Hand the work in the designated submission field of the assignment by the deadline.

Interns and trainees:

Interns do not yet have access to the Moodle system. Please place the task in your **personal folder** on the NAS ("\\itlabvi\\public"). Please set up a subdirectory "web" for this purpose.

Contents

Work order	2
Calculated processing time	2
The following rules apply to the submission of tasks	2
Official apprentices of the IT-L@B.....	2
Interns and trainees:	2
Responsive design.....	4
The CSS rule "display: flex "	4
The flex container	5
Flex-Direction	8
Behavior at the end of a row or column: flex-wrap	11
Flex items	14
The "order" property.....	14
The "flex" property.....	15
flex-basis	16
flex-grow.....	16
flex-shrink.....	20
Combination of flex-base, flex-grow and flex-shrink	21
Alignment of the Flex-Item.....	21
justify-content	21
align-items.....	23
align-self	25
Application examples of Flex technology.....	26
Structure of a website.....	26
Effect of "order"	27
Using Flex technology in conjunction with @media queries	29
Alignment of a menu block	31
Centered texts in a button	32
Questions for reflection.....	34

Responsive design

In the previous module (@media/@container queries), options were shown for querying the properties of the output medium and evaluating these for adapted formats using CSS. However, a real responsive design means that elements of the website are automatically according to the display options (e.g. viewport width).

In the "@media queries" module, it also became clear that there can be no general guideline for the conditions under which a website can be optimally displayed on a smartphone. This is due to the variety of devices and the fact that browser windows can be placed individually in their dimensions on the screen. The basic question is therefore which dimensions will allow the content of the individual elements to be displayed well or at least in an appropriate quality.

The HTML markup language has a certain basic responsiveness. For example, (unformatted) **block elements** have the width of the viewport or - in the case of nesting - of their parent element. If the maximum available width is reached, the page is wrapped, i.e. the following elements are moved to a new line. "Overwidth" block elements cause a scrollbar to appear.

So in the latter case you only see part of the content of the page. This is not really desirable.

The technique described below is intended to adapt the position of the elements to the dimensions of the viewport, whereby it is not necessary to change the order or content of the web page in the HTML code. A cleanly programmed web page (HTML) can therefore be very easily adapted using the "Flex" technology can be displayed responsively on different output devices. The (ideally very few) adjustments are made exclusively in the CSS code within @media containers.

The CSS rule "display: flex"

As already in Module 1, the CSS property "**display**" can be used to specify whether an element should appear as a block element (with line break) or as an inline element - without line break, integrated into the text flow. *Display* can also be used to create a "mixed form" in which an element appears like an inline element on the outside, i.e. is integrated into the regular text flow like a single letter, but can be formatted as a block element on the inside. You have seen this, for example, in the design of horizontally aligned navigation bars (example: `nav li {display: inline-block;}`).

The "*display*" property is also known from the option to show individual elements while other elements remain hidden. As a reminder:

```
display: none;
```

causes elements to be hidden⁽¹⁾. For example, all elements of type <article> can be completely hidden in this way. So that the page does not remain empty and the work of creating the <article> content is not in vain, a single <article> element - selected in the menu and linked via an ID - is shown using the dynamic pseudo class *:target*.

This use of the CSS property "*display*" is therefore already known.

¹As an alternative to "display: none;", the CSS property "*visibility: hidden;*" can also be used to display a element "invisible". However, "visibility" reserves the space originally required for the element on the website, which can result in large free areas appearing. Although this can be compensated for by additional formats in height and width (set both to zero), it requires additional programming effort.

However, the CSS property "*display*" can do much more. For example, it is possible to use "*display*" - in conjunction with other CSS properties that are coordinated according to precise rules - to completely restructure a page:

- Multi-column arrangement of elements on the page
- Changing the order of elements on the page
- General definition of distances between the elements
- General definition of the alignment of the elements

The CSS property "*display*" is used here to act as a container for displaying the elements integrated into it. In addition to the "Grid layout" - explained in the following module - this module the possible uses of the "Flex layout".

Note: In combination with the CSS media queries (*@media* and *@container*), the Flex layout can be used to implement responsive websites very easily and effectively, which work on all output devices without changes to the HTML code.

The flex container

A "flex container", which is declared as such with the CSS rule "*display: flex;*" and whose effect is described with other properties presented in this section, contains the so-called flex items (see below). The flex items are the "*child elements*" within the flex container. They are HTML elements that are arranged in the page according to the formats of the Flex layout. The rules for this arrangement can be, for example

- their width / height and by
- their order.

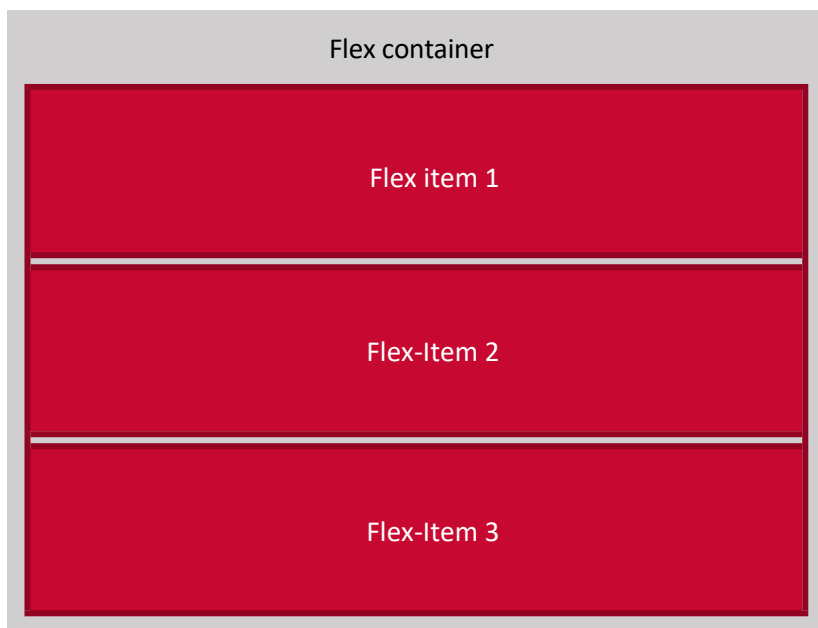


Figure 1: Example of a simple Flex layout: The Flex container here contains three HTML elements that represent the Flex item. Only the child elements, i.e. elements that are placed directly in the hierarchy as the first level within the flex container, are flex items.

First, let's look at the **flex container**. This is from an HTML element that encloses the subsequent flex item. Theoretically, the `<body>` element can already be declared as a flex container. All direct "child elements" (e.g. `<header>`, `<nav>`, `<main>`, `<footer>` etc.) are then the flex items. Flex technology can also be used to design menu bars.

can be used. Here, for example, the `<nav>` or `<ul>` element is declared as a flex container and its content is regarded as a flex item.

Note: Common research and training portals explain the Flex layout almost exclusively with `<div>` elements to which class attributes are assigned for differentiation. Reason: The training portals formulate **GENERAL** examples. In your training modules, however, you first use a predefined architecture to create a general understanding. Please work **EXCLUSIVELY** with the semantic HTML5 elements in your training.

The HTML structure sometimes used in training websites is as follows:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Flex1</title>
</head>
<body>
  <header>
...
  </header>
  <nav>
...
  </nav>
  <main>
    <article id="art1">
      <p>Article 1</p>
    </article>
    <article id="art2">
      <p>Article 2</p>
    </article>
    <article id="art3">
      <p>Article 3</p>
    </article>
    <article id="art4">
      <p>Article 4</p>
    </article>
  </main>
  <footer>
...
  </footer>
</body>
</html>
```

This structure will serve as the basis for the following explanations. The elements have sometimes only been assigned simple content (one paragraph element `<p>` each). To make them easier to read in the web browser

become visible, they are assigned individual background colors as well as a height and width using the following CSS code.

Tip: For the following experiments, these two codes can be saved in a sample file that be accessed again and again. Flex technology is not yet integrated here! The color scheme is of course optional. You are welcome to experiment with the heights and widths.

```
main {
  background-color: #faebd7;
}
article {
  background-color: gray;
  height: 100px;
  width: 100px;
}
article:nth-child(1)2 {
  background-color: #b76969;
}
article:nth-child(2) {
  background-color: #89c2e8;
}
article:nth-child(3) {
  background-color: #a5ecb1;
}
```

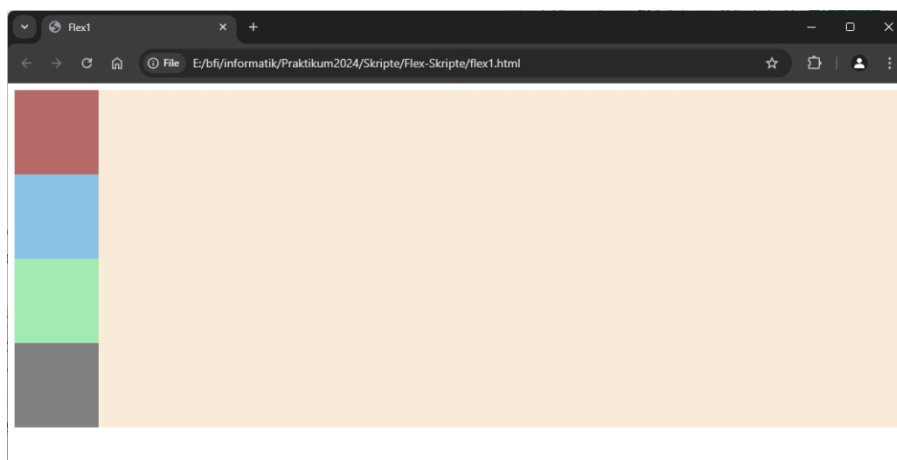


Figure 2: The HTML and CSS code described produces a result as shown the image. The vertical alignment is due to the block property of the <article> elements.

How Flex technology is used basically depends on the individual design. For example, it can be used for alignment and for the division of spacing in navigation bars, but also - in conjunction with the @media queries from the previous module - for the implementation of responsive designs with CSS.

²The *structural CSS pseudo-classes* such as :nth-child() will be covered in detail in a later module! You can find out more in advance in the publication selfhtml.org in the section CSS - Selectors.

Caution - Don't lose the overview! Flex technology is often used for all kinds of distributions of elements in a website, e.g. menu bar: Menu bar. However, since a responsive design - distribution and placement in the desired order of elements, depending on viewport widths and heights - is important in modern web design, flex technology is *ONE* way of adapting websites to different output devices. This will be the focus of this module.

Flex-Direction

The flex layout provides for a horizontal arrangement of the child elements³ (flex item) contained in the flex container from left to right without further specification. The flex items are arranged horizontally along the **main axis**.

It is also possible to arrange the Flex-Item vertically along the so-called **transverse axis** (from top to bottom). Both arrangement concepts (main axis or transverse axis) can also be implemented in the opposite directions (from right to left on the main axis and from bottom to top on the transverse axis).

The HTML code presented from page 6 shows four <article> elements that are integrated directly into a <main> container. They are therefore "child elements" of the <main> container. The container <main> should now serve as a **flex container**. The CSS code (from page 7) is supplemented slightly for this purpose:

```
main {  
    background-color: #faebd7;  
    display: flex;  
}
```

The CSS rule highlighted in red declares the <main> element as a **flex container**. This has immediate consequences, as in Figure 3. The same result can also be achieved by adding an additional CSS line:

```
main {  
    background-color: #faebd7;  
    display: flex;  
    flex-direction: row;  
}
```

Note: The default value for the alignment of the flex item within a flex container is "**flex-direction: row;**" - This line can therefore theoretically be omitted from the CSS code.

"**Row**" is the English term for a line. Accordingly, the alignments "row" and - as will be seen shortly - "row-reverse" are always alignments that refer to the main axis.

³ In web design, "family" terms such as "parent", "child" or in general "parentare used again and again. "Descendant" is used. Elements directly enclosed in a container are referred to as the "child element" (child) of the container, which is analogously the "parent container" (parent). Elements that are located within such a "child element" are referred to as descendants. They therefore do not directly follow the upper hierarchy of the top-level container.

To remember: <parent> <child> <descendant></descendant> </child> </parent>

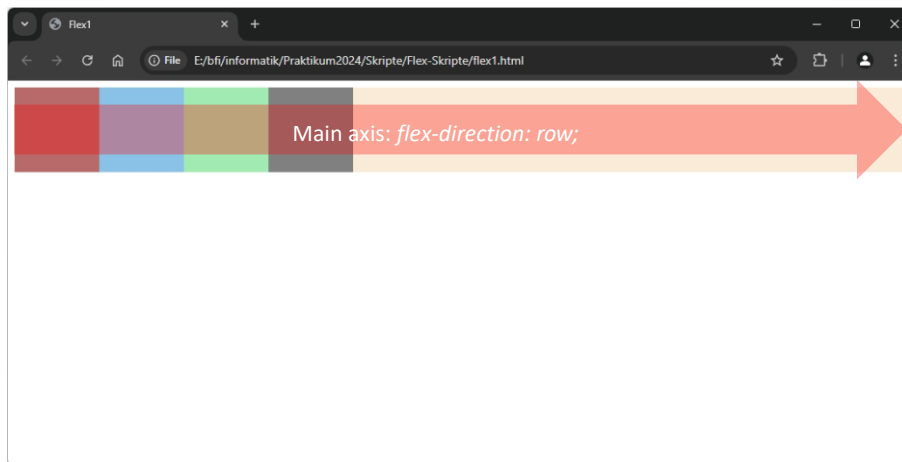


Figure 3: The `<main>` container surrounding the four `<article>` has been declared as a flex container. The arrangement of the (block) elements originally placed one below the other has changed. They now "flow" horizontally from left to right. This corresponds to the CSS rule "`flex-direction: row;`"

Accordingly, the alignment can still be defined on the main axis in the opposite direction. The CSS property required for this in the rule block of the flex container is: "`flex-direction: row-reverse;`". The CSS code is now changed in the line highlighted in red as follows:

```
main {
  background-color: #faebd7;
  display: flex;
  flex-direction: row-reverse;
}
```

Note: The term "reverse" means backwards or opposite. In the context of the CSS flex format, it means opposite to the normal writing or reading direction. In Central Europe is read from left to right (horizontal orientation) and the "reverse orientation" (row-reverse) is accordingly from right to left.

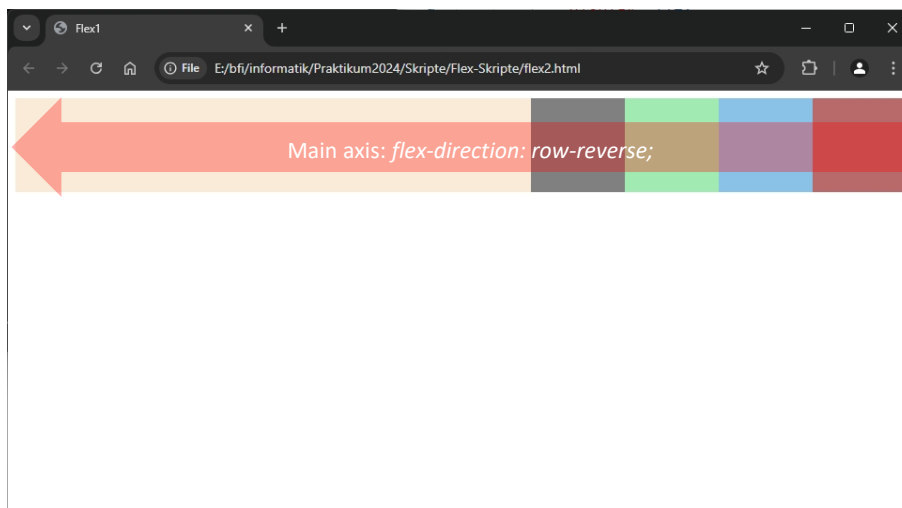


Figure 4: Still the same HTML code, but one line has changed in the CSS code: The flex direction is used here declared with "`row-reverse`". The alignment remains horizontal, but the elements are arranged from right to left.

In addition to the arrangement in relation to the main axis, the flex items can also be aligned vertically, i.e. in relation to the transverse axis. Here, the value for "flex-direction" is derived from the English word for column.

```
main {  
  background-color: #faebd7;  
  display: flex;  
  flex-direction: column;  
}
```

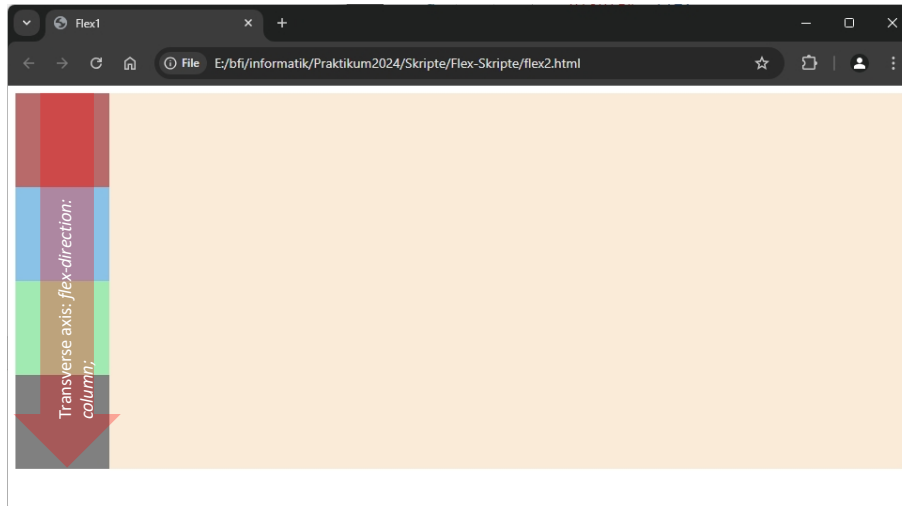


Figure 5: The HTML code is still unchanged. However, the value for the CSS property "flex-direction" is now: "flex-direction: column;"

Of course, the alignment can also be reversed from bottom to top. To do this, the flex-direction property is defined with "column-reverse".

```
main {  
  background-color: #faebd7;  
  display: flex;  
  flex-direction: column-reverse;  
}
```

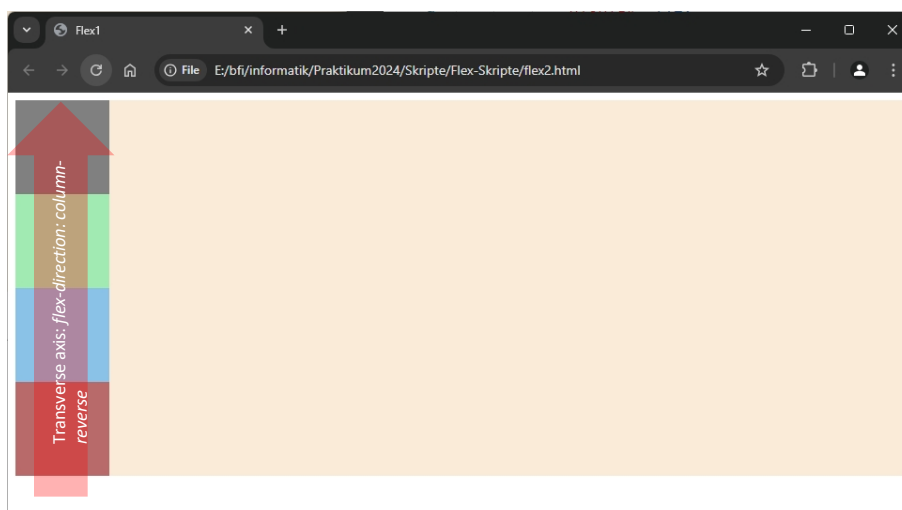


Figure 6: In line with the previous explanations in this module, the CSS line with the flex-direction declaration has been changed again. The assigned value is now "column-reverse".

Note: It is a "popular" spelling mistake: **column** must be written with "mn" at the end. become.

Behavior at the end of the row or column: flex-wrap

Beautiful! - You can now align elements horizontally or vertically from left to right or from bottom to top using Flex technology. However, the question now arises as to what happens if the viewport is too small, i.e. too narrow or too low. As an example for the following explanations, we will assume a horizontal alignment from left to right. To better illustrate the effect the HTML code will be expanded from four to eight `<article>` elements.

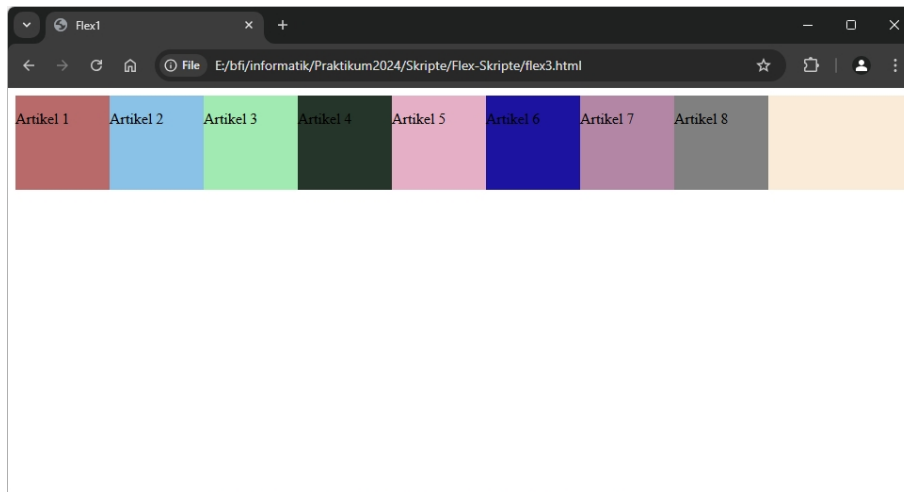


Figure 7: The HTML code for this representation corresponds to that of the previous examples. However, the following changes have been made: Instead of four, there are now eight `<article>` elements. The colors were freely assigned by an addition to the CSS code. As before, the `<article>` elements now contain simple paragraph elements `<p>` with a name (e.g.: Article 1).

Note: The new HTML and CSS code is not shown here. It is assumed in this module that you can add the new elements yourself without difficulty.

The viewport should now become narrower, so much so that the `<article>` elements no longer fit into the window in their originally defined width (the rule `width: 100px;` was entered in the rule block with the type selector `article`). An effect can be observed, as shown in Figure 8. The contents of the flex container are virtually squeezed together. This is not visually attractive. Another CSS flex property is used in the flex container: "flex-wrap".

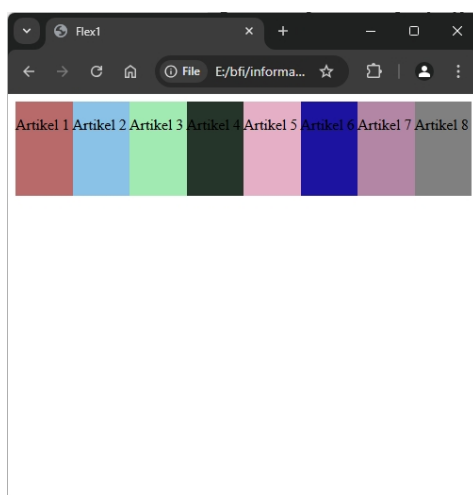


Figure 8: The viewport has been greatly reduced in size. However, by declaring it as a flex container (without any other additional formats), the contents of the `<main>` element are also compressed. This is not necessarily desirable!

The line marked in red in the following CSS code is inserted:

```
main {
  background-color: #faebd7;
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
}
```

The CSS rule "flex-wrap: wrap;" has the effect that the content of a flex container is wrapped into a new line when the limits of the viewport are reached. The result of the above CSS code is shown in Figure 9. The elements retain their original size, but move to a new line at the edge of the viewport.

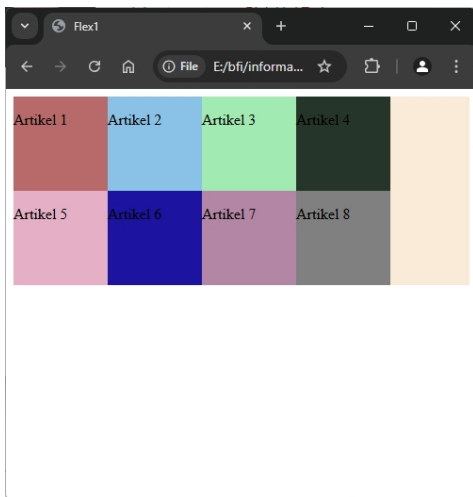


Figure 9: The width of none of the flex items is changed here. Instead, the CSS rule "flex-wrap: wrap;" causes a line break.

Of course, "flex-wrap: wrap;" also works in the reverse variant of alignment with the Main axis. Again, the CSS code needs to be adapted a little:

```
main {
  background-color: #faebd7;
  display: flex;
  flex-direction: row-reverse;
  flex-wrap: wrap;
}
```

Now note the progression of items 1 to 8 in Figure 10 and compare this picture with Figure 11!

In Figure 10, the reverse alignment on the main axis (CSS_Rule: "flex-direction: row-reverse;". This means that the reading direction from right to left dominates in a line (alignment to the main axis). In Figure 11, on the other hand, the reading direction row, i.e. along the main axis from left to right) has been retained, but the wrap has been inverted. The line is therefore wrapped from bottom to top, but the reading direction from left to right remains the same.

Attention: Please be careful if you use inverse values for the CSS properties flex-direction and flex-wrap.

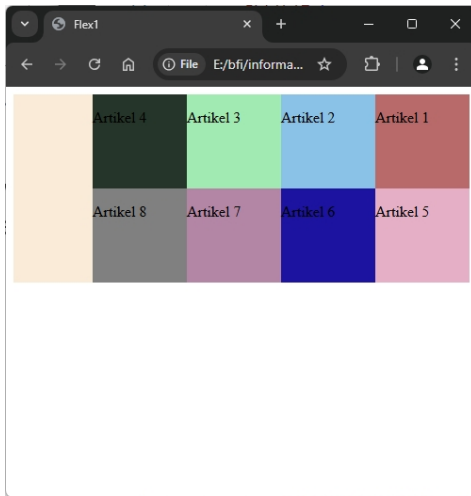


Figure 10: The rule "flex-direction: row-reverse;" arranges the elements from right to left, i.e. against the European reading direction. The rule "wrap: wrap;" forces a line break if the available space in the viewport is no longer sufficient to display an element in its defined width.

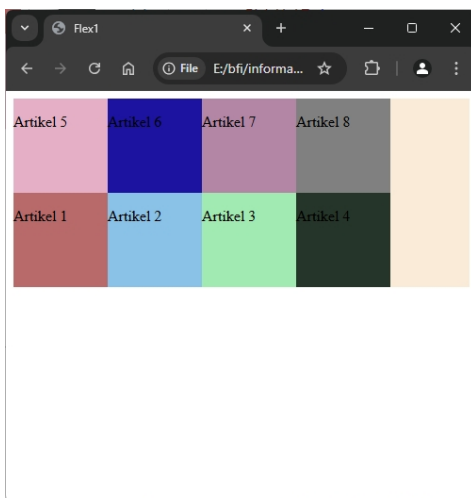


Figure 11: In this image, the reading direction has been set according to the general standard from left to right ("flex-direction: row;"). However, the wrap is "reverse", i.e. from bottom to top ("wrap-reverse;"). The first element therefore appears at the bottom left.

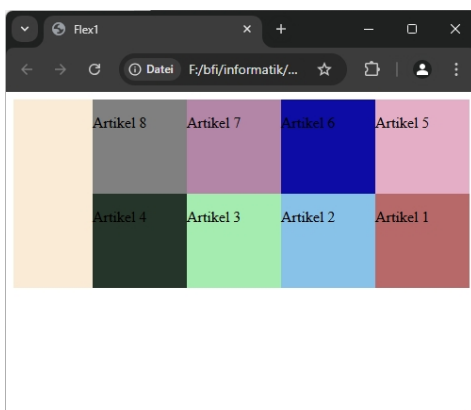


Figure 12: Both the reading direction ("flex-direction: row-reverse;") and the pagination are set to "reverse" ("wrap: wrap-reverse;"). The first element therefore appears at the bottom right. The reading direction is from right to left and the wrap is from bottom to top.

Flex items

The "order" property

In the examples, you can clearly see that the colored elements (here: <article>) are numbered in the order in which they are programmed in the HTML code⁽⁴⁾. If the order of these elements is to be changed, this could of course be done by moving the elements within the HTML code, but this is very unprofessional and also does not allow the order to be changed within a responsive design.

For a better overview, the HTML code from page 6 (only four <article> elements) and the CSS code from page 7 should be used again here. The <main> element is again the flex container with the following CSS rules:

```
display: flex;
flex-direction: row;
flex-wrap: wrap;
```

The aim now is to rearrange the order of the HTML elements within the flex container without changing the HTML code. This is very easy to do with a flex container using a new CSS property - order. Figure 13 shows the original result in the browser view. Immediately afterwards, Figure 14 shows how the CSS property "order" affects the individual flex items.

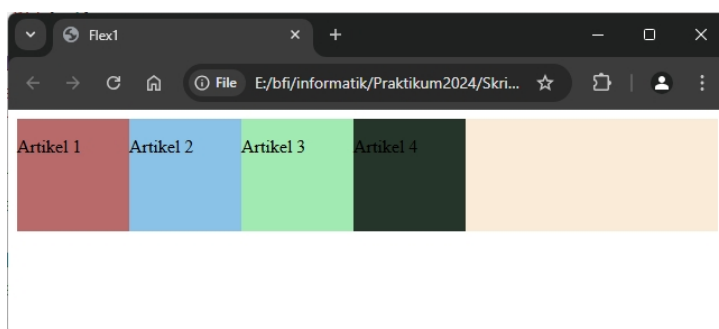


Figure 13: The first four articles in the order in which they are programmed in the HTML code: Reading direction is from left to right.

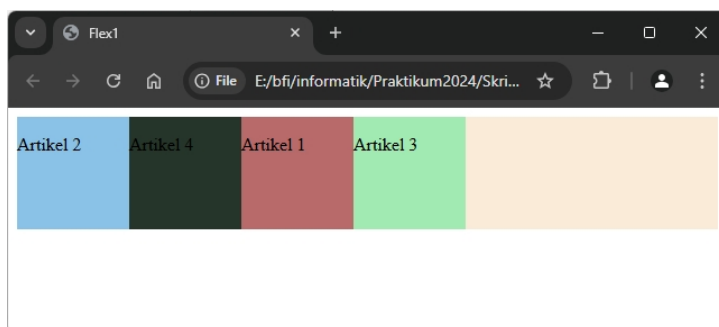


Figure 14: Each element has been assigned an "order property" in the CSS code, the only value of which is the order in the display.

Instead of in the order "1, 2, 3, 4" (Figure 13), as the elements were programmed in the unformatted HTML code, they now appear in the order "2, 4, 1, 3" in Figure 14. As already explained: The HTML code is unchanged.

⁴ The numbering is NOT automatic. These are merely lines of text within the <p> element, which is included in the <article>. The <article> element is the parent element of the respective paragraph.

Interesting are the additional CSS code lines that are included in the formats of the "*flex item*", i.e. in the elements that are the direct content of the flex container:

```
article:nth-child(1) {  
    background-color: #b76969;  
    order: 3;  
}  
article:nth-child(2) {  
    background-color: #89c2e8;  
    order: 1;  
}  
article:nth-child(3) {  
    background-color: #a5ecb1;  
    order: 4;  
}  
article:nth-child(4) {  
    background-color: #263529;  
    order: 2;  
}
```

The CSS property "*order*" determines the order of the flex items within the flex container based on the programmed values. This makes it possible, for example, to arrange elements in a responsive design according to their importance for the viewer if the viewport is too narrow. It is therefore conceivable that a page an advertising block that is placed on a wide viewport to the left of the main part. However, the visitor to the page will quickly lose interest in the publication with a narrow viewport if this advertisement is presented on the smartphone before the actually interesting content and the visitor has to scroll through the page. The CSS property "*order*" can be used in a flex layout to ensure that the relevant content appears in a better position.

The "*flex*" property

First of all, possible misunderstandings should be avoided: In the code block on page 8, the term "*flex*" was used as the **value** for the CSS property **display**. With the resulting CSS_rule "**display: flex;**", the container in question is declared as a "flex container" for the child elements it contains, which are henceforth also referred to as *flex items*.

Attention: In the CSS rule "**display: flex;**", "display" is the CSS property and "flex" is its "value"!

There is also the **CSS property** "flex", which is placed in the CSS code of the *flex container* and which also has a direct effect on the width of the flex item and its behavior when the viewport changes. Up to three values can be assigned to the "flex" property:

- flex-basis
- flex-grow
- flex-shrink

These properties can be used to define the behavior of a flex item when the viewport size changes, starting from a base size.

flex-basis

The "flex-basis" is the initial width of a flex item within the flex container. The common units of measurement for width declarations can be used as values. Examples: px, em, %, vw etc.

In the following example, the four <article> elements, which are located in the flex container (<main>), are defined with different flex-basis values (100px, 200px, 300px and 400px) in their width as the base width. The result is shown in Figure 15. Alternatively, the respective width formats could of course also have been defined with the CSS property "width". The result in the display would be identical, at least in Figure 15. However, the following CSS properties "*flex-grow*" and "*flex shrink*" would not work.

The following CSS code (excerpt with only two formats) shows the formatting with the "*flex-basis*" property and individual widths.

```
article:nth-child(1) {
    background-color: #b76969;
    flex-basis: 100px;
}
article:nth-child(2) {
    background-color: #89c2e8;
    flex-basis: 200px;
}
...
```



Figure 15: Result of the described flex layout with different values for flex-basis (100px, 200px, 300px, 400px)

flex-grow

If the example in Figure 15 has been reprogrammed, you will notice that the elements do not change (become wider or narrower) even if the width of the flex container is changed. Instead, a line break may be set according to the property of the flex layout. Elements that no longer fit completely into the width of the page are placed in a new line⁵, as shown in Figure 15. This is not always desirable.

The CSS property "*flex-grow*" (as well as "*flex-shrink*") can provide a solution here.

The CSS property "*flex-grow*" defines the ratio in which the dimensions of the relevant element are changed (also enlarged) when the flex container is *enlarged*.

An example will show how the *flex-grow* property works, how precisely the widths (or heights in the case of a column-oriented flex container) can be calculated and what influence the property has on a wrap (container is wider than the sum of the elements it contains).

⁵ In the case of a column-oriented page, the page break naturally occurs in a new column.

has. In Figure 16, all article elements have a width of 100px each. The width of the flex container (<main> element), in which the articles have been placed, is set to 400px⁶. In this case, this also corresponds to the viewport width, because no width was set for the <main> container (no width property). It is very easy to observe that all articles are actually the same width, although the CSS code, as shown on page 18, provides different values for flex-grow. The "flex-grow" properties in the respective containers must therefore be observed. These are highlighted in red in the code shown.

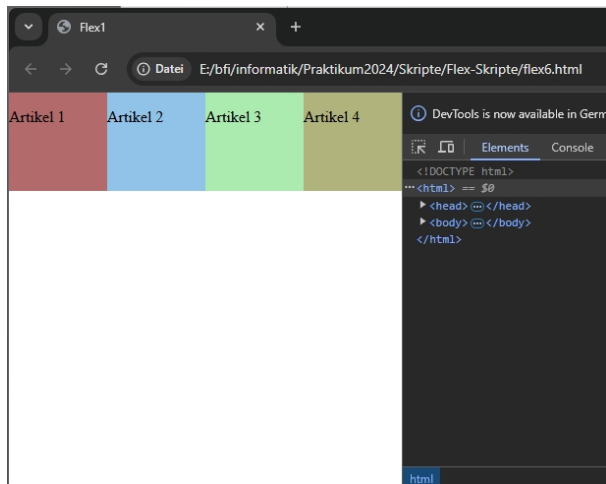


Figure 16: In this example, the width of the flex container (here: the <main> element) was adjusted to exactly 400px (4 x 100px of the individual elements in width). All elements in the example have exactly the same width (100px each).

Note: How do you find out the width of an element in a web page? - The developer tools of the web browser can help here: Right-click in the web page and select *Inspect* in the context menu. You can also access the developer tools via shortcuts:

- [Umschalt]+[Strg]+[I]: Last used developer tool
- [Umschalt]+[Strg]+[J]: Console
- [Umschalt]+[Strg]+[C]: Elements (possibly combined with the

formats) The areas of the developer tools can be changed via the tabs.

Now search for the element to be viewed in the code in the developer window. The element is highlighted in the web page and its dimensions are displayed in a tooltip (see article#art2 in Figure 17).

```
main {
  background-color: #faebd7;
  display: flex;          /* <main> is the flex container */
  flex-direction: row;
  flex-wrap: wrap;
}
```

⁶) Open the developer tools of the browser and move the visible area of the page together or apart with the mouse until a width of 400px is visible at the top right (in the page).

```

article {
  background-color: gray;
  height: 100px;
  flex-basis: 100px;    /* The <article> element has a base width of 100px */
}
article:nth-child(1) {
  background-color: #b76969;
  flex-grow: 1;
}
article:nth-child(2) {
  background-color: #89c2e8;
  flex-grow: 3;
}
article:nth-child(3) {
  background-color: #a5ecb1;
  flex-grow: 2;
}
article:nth-child(4) {
  background-color: #b1b47d;
  flex-grow: 1;
}

```

Although the "flex-grow" property has been assigned different values, all articles here are (still) the same width (100px each). The code remains absolutely unchanged for the following experiment! Now, however, the width of the flex container (here still the <main> element) is to be set to 800px. A completely new appearance appears (see Figure 17). The elements are considerably different in width.



Figure 17: The same code (both HTML and CSS), but the **viewport width** is set to 800px.

If you hover over the element to be examined (here: article 2) in the developer tools in the "Elements" area, you can see symbolically how wide the element originally was in its basic setting (by flex-basis) and how much the element has now increased in size due to the doubling of the width of the flex container. It is noticeable that item 2 has not grown to three times its original width, even though the rule "**flex-grow: 3;**" has been set.

Let's take a mathematical look at the CSS property "flex-grow" (note: *please allow some time for this to really understand the principle!*)

As seen, the following CSS rules form the basis (only the rules that are actually relevant here are listed):

```
main {... flex-base: 100px; ...}
article:nth-child(1) {... flex-grow:
1;...} article:nth-child(2) {... flex-
grow: 3;...} article:nth-child(3) {...
flex-grow: 2;...} article:nth-child(4)
{... flex-grow: 1;...}
```

Expressed in a formula, the situation is as follows:

$$100 \cdot (1 + 1x) + 100 \cdot (1 + 3x) + 100 \cdot (1 + 2x) + 100 \cdot (1 + 1x) = 800$$

Green is therefore the flex base and the **flex-grow values** are displayed in color. "x" is a factor to be calculated here, which represents the basic value by which the space gained by enlarging the viewport is filled. However, because the flex base always remains the same (the enlargement of the element is added to this), the brackets must contain "**1+ flex-grow * factor x**". The current width of the viewport is shown to the right of the equals sign.

Dissolve the brackets:

$$100 + 100x + 100 + 300x + 100 + 200x + 100 + 100x = 800$$

Summary of the values without factor x and summary of the x-values:

$$400 + 700x = 800$$

To reiterate: The value "400" highlighted in red on the left-hand side of the equation corresponds to the sum of all flex base values in the flex container. The value "800" highlighted in blue on the right-hand side of the equation corresponds to the current viewport width. The green
The emphasized term "700x" is made up of the factor for the enlargement, which is still to be calculated, and the product of the flex-grow sum with the flex base. We can remove the value 400 from the equation by subtracting it on both sides.

$$700x = 400$$

"x" should on the left-hand side alone. We divide both sides of the equation by 700.

$$x = \frac{400}{700}$$

Determine the value for x (here both sides divided by 700=> Target left: 1x):

$$x = 0,57142$$

Brain teaser:

Mathematically gifted people may be able to derive a universal formula for different flex-grow sums and any viewport width.

The calculated x-value applies to the example used here with a flex container width of 800px. The **x-value** calculated in each case is the factor to be multiplied by the value specified for the respective flex item with "flex-grow". Compare the values in the table with those you observe on the website.

Base width	x-factor	Flex-grow value	Final element width
100px	0,57142	1	157,142
100px	0,57142	3	271,426
100px	0,57142	2	214,284
100px	0,57142	1	157,142
Total width (width of the Flex container)			799,994px (~ 800px)

flex-shrink

The previous section showed how the contents of the flex container - i.e. the flex items - react to an enlargement of the container if a special factor (flex-grow) is declared for each flex item. An understanding of this section is assumed in the following.

The CSS code is changed a little. The colored lines are important. Flex-wrap formatting should not be used.

```
main {
  background-color: #faebd7;
  width: 100vw;
  display: flex;
  flex-direction: row;
  flex-wrap: wrap; /* Attention: Do not use flex-wrap!!! */
}

article {
  background-color: gray;
  height: 100px;
  flex-base: 100px;
  flex-grow: 1;
}

article:nth-child(1) {
  background-color: #b76969;
  flex-shrink: 1;
}

article:nth-child(2) {
  background-color: #89c2e8;
  flex-shrink: 3;
}

article:nth-child(3) {
  background-color: #a5ecb1;
  flex-shrink: 2;
}

article:nth-child(4) {
  background-color: #b1b47d;
  flex-shrink: 1;
}
```

Note: flex-wrap versus flex-shrink

The CSS property flex-shrink is effective if the width of the flex item is also adjusted by reducing the size of the flex container. However, if the flex-wrap property is used in the formats of the flex container, flex items that no longer fit into the available width of the container are simply wrapped in a new line. Flex-shrink then has virtually no effect.

The calculation of the element widths of the flex item when shrinking under the influence of flex-shrink is similar to flex-grow. However, a subtraction is carried out within the bracket, as the flex item should also become narrower.

$$100 \cdot (1 - 1x) + 100 \cdot (1 - 3x) + 100 \cdot (1 - 2x) + 100 \cdot (1 - 1x) = 270$$

$$100 - 100x + 100 - 300x + 100 - 200x + 100 - 100x = 270$$

$$400 - 700x = 270$$

$$-700x = -130$$

$$-130$$

$$x = \frac{-130}{-700}$$

$$x = 0,185714$$

Base width	x-factor	Flex-shrink value	Final element width
100px	0,185714	1	81,42
100px	0,185714	3	44,29
100px	0,185714	2	62,86
100px	0,185714	1	81,42
Total width (width of the Flex container)			269,99px (~ 270)

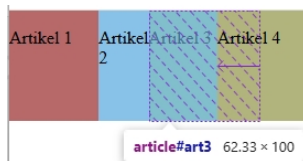


Figure 18: The viewport now has a width of 270px. Due to the CSS property `flex-shrink`, which has been defined very differently for each flex item, the article elements have also been reduced in size to varying degrees.

Combination of flex-base, flex-grow and flex-shrink

The CSS property "flex" offers the option of declaring all the formats described above in just a single CSS line. The following line (please note the order: grow - shrink - base)

```
flex: 1 3 100px;
```

corresponds to the CSS

```
flex-grow: 1;
```

```
flex-shrink: 3;
```

```
flex-base: 100px;
```

Note: All three properties (flex-grow, flex-shrink and flex-basis) can be set in the CSS property "flex" can be declared together (please do not forget spaces between the values!).

Alignment of the Flex-Item

In conjunction with @media queries, Flex technology makes it possible to react specifically to changes in the dimensions of the viewport. A website can thus be displayed on a smartphone as well as on a desktop computer monitor in a structure that is optimized for the device.

It is also possible to align the flex item within a flex container according to certain criteria on the (horizontal) main axis and on the (vertical) cross axis.

The property for aligning the flex item within a flex container to the main axis is called `justify-content`. The property for aligning the flex item to the transverse axis is `align-items`.

justify-content

The `justify-content` property is declared in the rules for the flex container and applies to the horizontal alignment of all flex items within this container.

The following CSS code was for the illustration. The corresponding HTML code corresponds to the examples of this module with four flex items (four <article> elements within the <main> element).

```
main {
  background-color: #faebd7;
  width: 100vw;
  height: 300px;
  display: flex;
  flex-direction: row;
  justify-content: center;
}
article {
  background-color: gray;
  height: 100px;
  flex-basis: 100px;
}
article:nth-child(1) {
  background-color: #b76969;
}
...
article:nth-child(4) {
  background-color: #b1b47d;
}
```

The following values can assigned:

- **center** The flex items are aligned horizontally centered on the main axis. The elements are therefore evenly distributed to the right and left of the transverse axis.
- **flex-start** The flex items are on the main axis.
- **flex-end** The flex items are aligned to the right of the main axis
- **space-between** The flex items are aligned at equal distances from each other along the main axis. However, the first flex item is placed directly to the left and the last flex item directly to the right edge of the flex container with no space between them.
- **space-around** The space between the flex items is basically identical. There is also a space to the left and right edge of the flex container, but this is only half the size of the space between the flex items.
- **space-evenly** The "space-evenly" value has a similar effect to "space-around", but the distances to the left and right edges of the flex container are just as wide as between the individual flex items.

The following illustrations show the results if the values for *justify-content* are changed accordingly in the line highlighted in red in the previous code example.



Figure 19: Alignment along the main axis with **justify-content: center**;

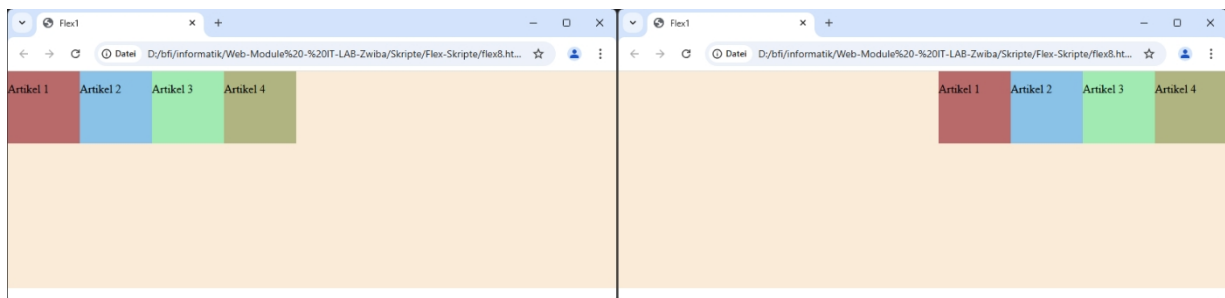


Figure 20: Alignment along the main axis with **justify-content: flex-start**; (left image) and **justify-content: flex-end**; (right picture)

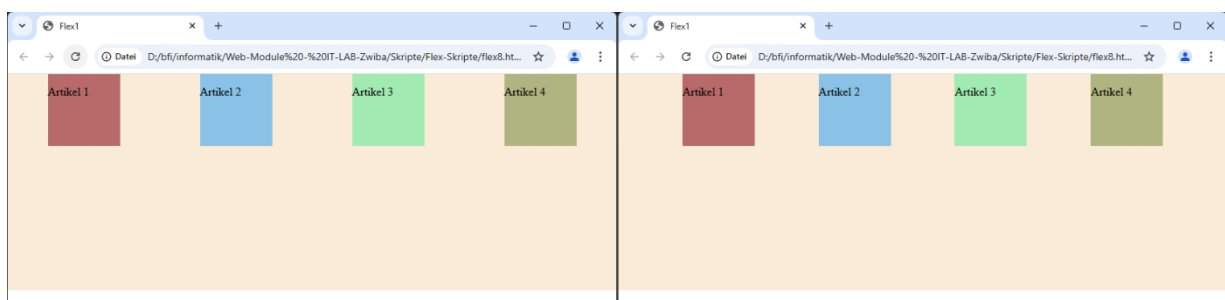


Figure 21: Difference between **justify-content: space-around**; (left image) and **justify-content: space-evenly**; (right image). In both cases, the spacing between the flex items is always identical. In the right-hand image, however, the distance to the edges also corresponds to the distance between the flex items. This is not the case with **"space-around"**.



Figure 22: Unlike the two previous images, the **justify-content: space-between**; rule does not have any spacing to the right or left edge of the container. However, the spacing between the flex items is identical.

align-items

The CSS code from page 22 should be used again. However, the line marked in red is replaced by the CSS property **"align-items"** with one of the following values. This is the alignment along the (vertical) transverse axis.

- **center** The flex items are aligned vertically centered on the transverse axis. The elements are therefore distributed evenly above and below the (horizontal) main axis. The effect is therefore a "vertical centering".

- **flex-start** With the rule "*align-items: flex-start;*" in the flex container format, the flex items are aligned to the top edge of the container and, in the case of a wrap, to the bottom.
- **flex-end** With the rule "*align-items: flex-end;*" in the flex container format, the flex items are aligned to the bottom edge of the container and to the top in case of a wrap.
- **stretch** With the rule "*align-items: stretch;*", the flex item fills the available height within the flex container. However, it is important to ensure that no height has been assigned to the elements defined as flex items, as this would otherwise be dominant over "align-items".
- **baseline** With the rule "*align-items: baseline;*", the flex items take on the height they need to display their content. The reference is the baseline of the last line of content. This can also be beyond the declared height of the flex container. Here too, the elements that represent the flex item must not be assigned a height in their formatting, as this would otherwise be dominant over the align-items property in the flex container.
- **normal** Default value if no rule is declared for align-items. The behavior corresponds to "align-items: stretch;" with the restrictions already explained.

Note: No height may be defined for the flex item for the values *baseline* and *stretch*, because the height declaration for the respective elements dominates and suppresses the effect of align-items.

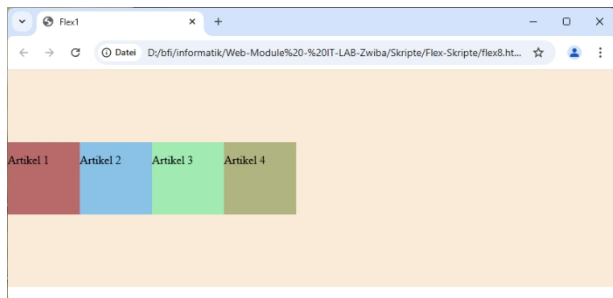


Figure 23: If only the rule "*align-items: center;*" is written into the format of the flex container, the elements are aligned vertically centered.

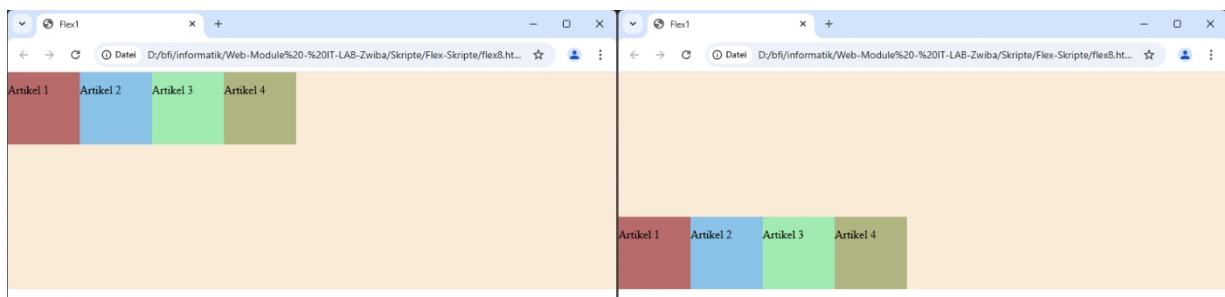


Figure 24: With the rule "*align-items: flex-start;*", the elements are aligned vertically at the top edge of the flex container. Any wrapping takes place at the bottom. With the rule "*align-items: flex-end;*", on the other hand, it is the other way round: the vertical alignment starts from the base line of the flex container and moves upwards.

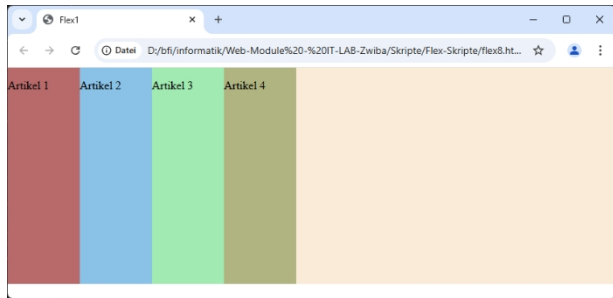


Figure 25: The rule "align-items: stretch;" causes the height of the flex item to the full height of the flex container. No fixed height may be defined in the formats of the respective flex item itself!

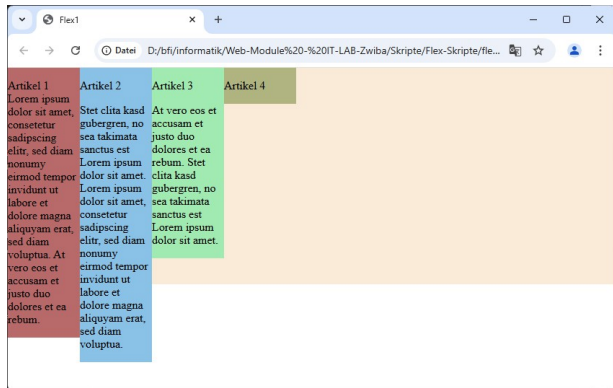


Figure 26: The rule "align-items: baseline;" defines the height of the respective flex item based on its content. No fixed height may be defined in the formats of the respective flex item itself!

align-self

The align-items property, which is declared in the format of the flex container, is used to define the alignment of *all* flex items *within the flex container*. The **align-self** property, which is **only** used **in the affected flex item** (NOT in the flex container), can be used to define exceptions for individual elements.

An example (only an excerpt from the entire code):

The flex container (here: <main>) is formatted with the following code (excerpt):

```
main {
  display: flex;
  flex-direction: row;
  align-items: flex-start;
}
```

The flex item with the content "Article 2" should be an exception in its alignment and appear at the bottom of the flex container. The line highlighted in green can therefore be found in the CSS code block of this element.

```
article:nth-child(2) {
  align-self: flex-end;
}
```

Try other exceptional formats too!



Figure 27: The element with the content "Article 2" was formatted with "align-self: flex-end;". Although the format in the flex container provides for alignment according to the rule "align-items: flex-start;", it therefore appears at the bottom of the flex container.

Application examples of Flex technology

Structure of a website

Figure 28 to Figure 32 show a web page based on the following HTML code (extract from the complete code).

```
<html>
<head> ... </head>
<body>
  <header>
...
  </header>
  <nav>
    <ul>
      <li><a href="#art1">Article 1</a></li>
      ...
    </ul>
  </nav>
  <main>
    <article id="art1">
      ...
    </article>
... <!-- more articles --> ...
  </main>
  <footer>
    ...
  </footer>
</body>
</html>
```

In addition, the following CSS formats apply (Attention: This is only an excerpt from the code! Dynamic pseudo-classes, button formats and color schemes etc. are not listed here in the code).

```
body {  
  
    display: flex;  
  
    flex-wrap: wrap;  
  
    flex-direction: row;  
  
}
```

The `<body>` of the page is also the *flex container*. This means that its direct *child elements* (here: `<header>`, `<nav>`, `<main>` and `<footer>` are the *Flex item*. The situation is different for the elements `<article>` (content of the container `<main>`), `<ul>`, `<li>` and `<a>` (content of the container `<nav>`) and, of course, the other content elements of the flex item. Although they are all *descendants* of the `<body>` container, they are not direct child elements and therefore not flex items.

The `<body>` element was made a flex container in the original code with the *"display"* property. The other properties provide for a wrap at the edge of the container and horizontal alignment from left to right.

```
header {  
  
    order: 1;  
  
    flex: 100%;  
  
    height: 80px;  
  
}  
  
nav {  
  
    order: 2;  
  
    flex: 24%;  
  
    margin-right: 5px;  
  
}  
  
main {  
  
    order: 3;  
  
    flex: 75%;  
  
}  
  
footer {  
  
    order: 4;  
  
    flex: 100%;  
  
}
```

Effect of "order"

The *"order"* property already discussed (see page 14 onwards) the order in which the elements appear on the website. As will be seen, it does not matter in which order the elements were actually programmed. In this case, the following order is defined.

1. `<header>`: With the additional property `"flex: 100%;"`, the element is assigned the full page width. This means that the following element is automatically placed in the next line⁽⁷⁾. The height of the header is also set to 80px.
2. `<nav>`: This flex item is assigned just under a quarter of the width of the flex container with `"flex: 24%;"`. As there is no other formatting, the navigation area appears on the left edge of the container.
3. `<main>`: There is still plenty of space to the right of the `<nav>` container, as this should only occupy 24% of the flex container width. The `<main>` container was also not assigned the full width of a line with `"flex: 75%"`. However, it fits very well next to the previously placed `<nav>` container, so that a two-column layout is created here.
4. `<footer>`: The footer is placed in the fourth position and, as with the header, is assigned the full width of the flex container (`flex: 100%;`).

The result is an image as shown in Figure 28. As already indicated, different background colors have been assigned to the elements. As the handling of the basic CSS formats can now be assumed, the lines of code required for this are no longer listed in this script.



Figure 28: The `<body>` has been declared a flex container. Flex items (in brackets the assignment of the order with the "order" property) in this example are the elements `<header>` (order: 1;), `<nav>` (order: 2;), `<main>` (order: 3;) and `<footer>` (order: 4;).

can experiment a little! In Figure 29, the formats of the elements `<nav>` and `<main>` and swapped the order of these two elements defined by the "order" property. The menu block is now in the third position within the flex container, the main part `<main>` is now the second position. Both elements still fit in one line, which only changes their position on the page.

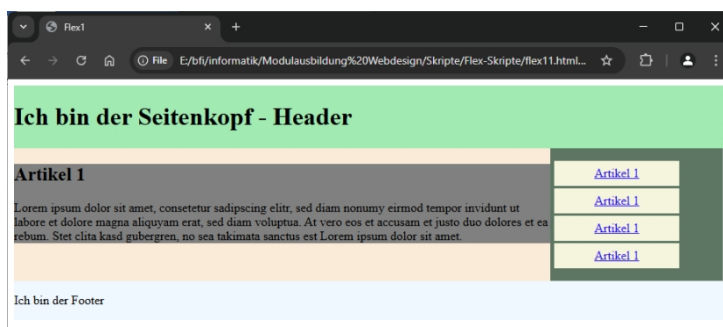


Figure 29: The values of the order property have been swapped for the `<nav>` and `<main>` elements. This is why the menu now appears on the right-hand side. The rest of the HTML and CSS code is unchanged.

The website in Figure 30 somewhat peculiar. The third position was created using the property "order", the `<footer>` element is set, which requires 100% of the flex container width.

⁷ As a reminder: The flex container was formatted with `"flex-direction: row;"` and `"flex-wrap: wrap;"`.

This means it no longer fits next to the menu block and has to move to the third line. The `<nav>` container, on the other hand, spreads over the entire line, as can be seen from the dark green background color. The same applies to the `<main>` container, which has been moved to position 4 here.

Of course, the example shown in Figure 30 is not ideal because a lot of space on the page remains empty and is therefore wasted.

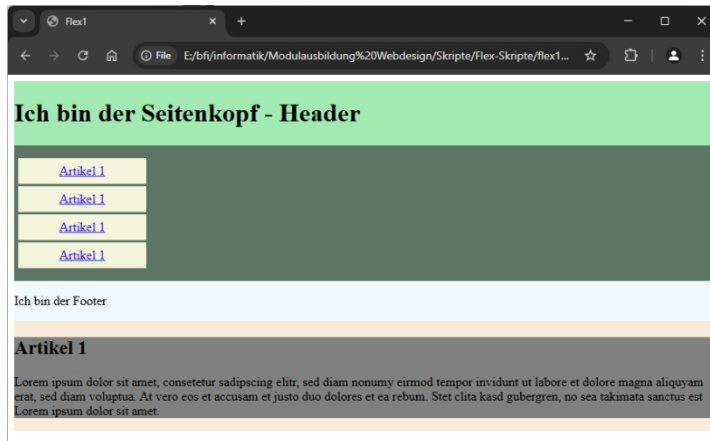


Figure 30: The order of the flex item has been chosen very awkwardly here, which means that a lot of space is wasted on the page next to the menu. The reason for this is the format of the flex item `<footer>` (placed here with "order: 3;" in front of the `<main>` container), which was set to the full width of the flex container with "flex: 100%". This means that the element no longer fits into the free space to the right of the menu block.

A final example should once again illustrate the effect of the "order" property. It should be emphasized that no changes were made to the HTML code (not even in the order of the elements). Only the values of the "order" properties were changed in the CSS code. In Figure 31, the `<header>` was assigned the place at the bottom of the page with the rule "order: 4;".

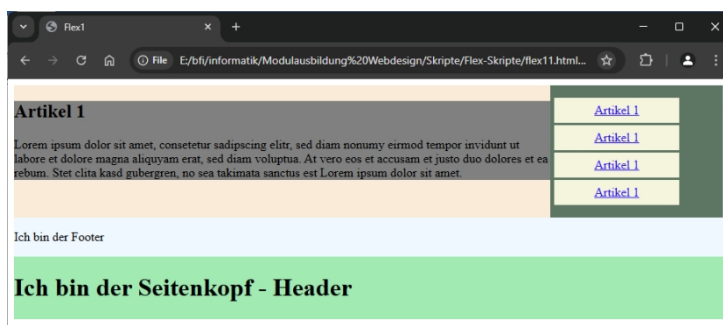


Figure 31: This arrangement of the website is completely "slapstick", but only due to the change in the order values. Here, the `<header>` is suddenly in 4th place in the order. The order values of the other elements have also been changed in this example.

Use of Flex technology in conjunction with @media queries

In *Module 13 - Responsive websites - Media queries*, query options for the height and width of the viewport were introduced. The `@media` query can be used to define special rules that should apply if the width of the viewport is possibly too narrow and the elements no longer fit next to each other on the page in the desired way. In such cases, the procedures shown in this module make it possible, for example, to change the order in which the elements are displayed or their width. In addition, further CSS formats can be defined, which only apply within the framework of condition defined with the `@media` query.

Important: Please always place the `@media` queries at the end of the CSS code. Also (and especially!) make sure that you only write or change lines of code in this block that are to apply in this particular exceptional case. All other formats remain valid. It is therefore essential that you avoid copying the entire CSS code you have previously written into the `@media` blocks! This would make your code confusing and far too long. In the event of a change, there is a risk that you will no longer use uniform formats on the page in different situations.

In the following example, only the formats of three elements need to be adjusted in order to achieve the desired layout (all elements 100% of the flex container width, horizontal menu in penultimate position).

```
@media (max-width: 750px) {  
  nav {  
    order: 3;  
    flex: 100%;  
    margin-right: 0px;  
    text-align: center;  
  }  
}
```

Most of the work will be invested in redesigning the menu. In future, it will appear in third place - after the `<header>` and after the main part (`<main>`). All elements should take up the full width of the viewport. This is ensured with the rule `"flex: 100%;"`, which should now (viewports $\leq 750\text{px}$) apply to both the `<nav>` element (for viewports $>750\text{px}$: 24%) and the `<main>` element (for viewports $> 750\text{px}$: 75%). New values must therefore be assigned here for the `"flex"` property.

The value for `"margin-right"` is also changed for the `<nav>` element. This is to `0px`. The inline content is aligned horizontally in the middle with the rule `"text-align: center;"`. This is possible because the rule `"display: inline-block;"` was set in the rule block with the selector `"nav li"` - this was not yet included in the regular code of the example. The list items thus appear like "text characters", i.e. *without a line break*. So that the buttons do not "stick" directly to each other, a *margin* is set on the right-hand side.

```
nav li {  
  display: inline-block;  
  margin-right: 3px;  
}
```

With the two CSS rule blocks described above, the menu was placed horizontally across the full page width below⁸ the main section. Finally, two adjustments to the formats of the `<main>` element are required: it should move to the second position after the `<header>` and be presented in the full width of the viewport. This is done by the following CSS block.

```
main {  
  order: 2;  
  flex: 100%  
}  
}
```

⁸Of course, this is an example to illustrate the effect of the combination of Flex-Layout and `@media-Query`. The menu normally belongs before or after the header, but in any case in an easily accessible place on the website. Some web designers use two identical menu blocks before and after the main body if the page is particularly long.

To check the formats, the (updated) page is now called up in a web browser and dragged narrower or wider on the page. At the 750px limit of the viewport width⁽⁹⁾, the order and arrangement of the elements should change and the page should appear approximately as shown in Figure 32.

Note: In this example, the horizontal alignment of a menu for the `<li>` element in the menu is achieved with the rule `"display: inline-block;"`. Horizontal centering is also achieved with the rule `"text-align: center;"`. What becomes difficult with this method is the direct vertical centering of the elements. The alternative shown below may solve this problem.

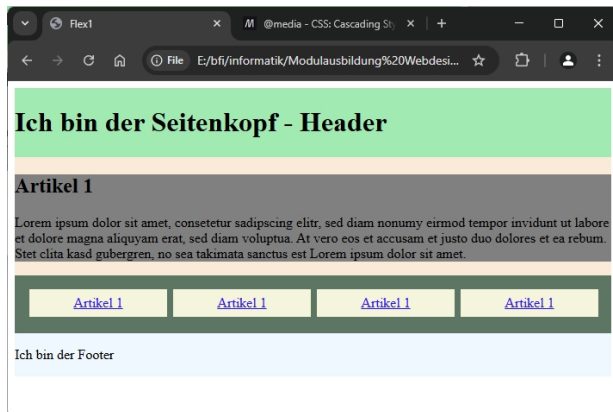


Figure 32: Up to a maximum viewport width of 750px, rules defined in an `@media` block are used to change the individual formats of the flex item elements and their order. The page can therefore react responsively to changes in the screen width.

Alignment of a menu block

By declaring an HTML element as a *Flex container*, the *child elements* included in it automatically become *Flex items*. The default setting for Flex is that all elements are aligned from left to right. But beware: Without further formatting, there is no wrap at the edge of the flex container! If no fixed width has been assigned to the buttons, their width will even be compressed (without wrapping).

If, for example, a menu is not to be aligned vertically - as shown in the first module - but horizontally, this can be implemented very easily by declaring the `nav ul` element as a flex container.

```
nav ul {
  list-style: none;
  padding-left: 0px;
  display: flex;
}
```

⁹ This value applies to the example shown here!

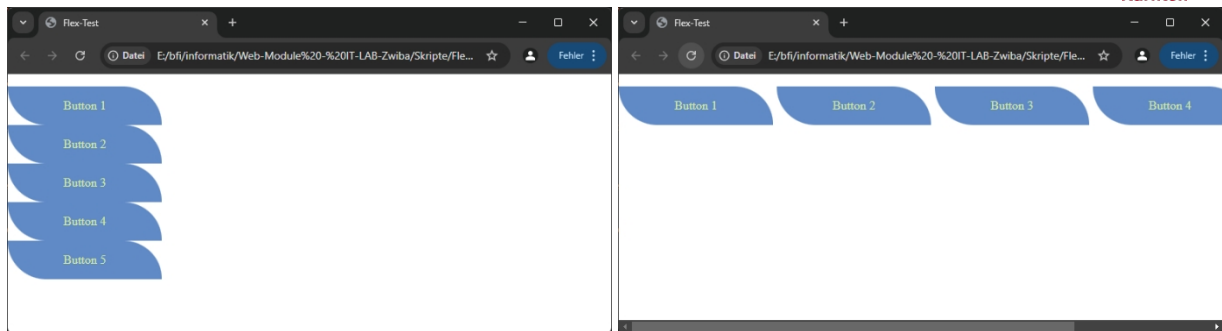


Figure 33: Left: No flex container has been defined. Right: The `<ul>` element (within the `<nav>` container) was declared a flex container with the rule "display: flex;". The child elements included in it (in a list, these are the list items, `<li>`). Are now the flex items. They appear in horizontal alignment after the basic formatting of a flex container. As can be seen, however, this happens without a break at the edge of the container (the buttons have been assigned a fixed width).

Note: already shown in this module, a wrap at the edge of the flex container can be forced with the rule "flex-wrap: wrap;" (see Figure 34).

Minor "blemishes" still need to be corrected, such as the spacing of the elements. In general, however, it would be advisable to choose a better design for narrow viewports where there are breaks in the menu. Reminder: Appropriate decision criteria can be defined with the @media queries and suitable formats can be entered in their code blocks.



Figure 34: The format of the flex container (here: `nav ul`) is supplemented by the rule "flex-wrap: wrap;". Now the flex items that extend beyond the edge of the container are wrapped in a new line.

Centered texts in a button

Assume an HTML page with a small menu as described in the first modules. The light gray CSS lines of the following code describe part of the transition from the list to buttons, which are to be used here (selector: `nav li`) in horizontal alignment. The rule block with the selector `nav a` formats the hyperlink element `<a>` within the list item as a button. This is already known from the previous lessons.

```
nav ul {
  list-style: none;
  padding-left: 0px;
}

nav li {
  display: inline-block;
}
```

In the red line, the `<a>` element is defined as a flex container. The `<a>` element only contains a text here, i.e. no further child element that would have to be declared as a flex item.

Nevertheless, the CSS properties *justify-content* and *align-items* can be used sensibly to ensure a clean horizontal AND vertical alignment with the respective value "center". The text - no longer wrapped in other elements - takes the place of the flex item here.

Watch out! Problems with the display are also to be expected here if single-line and double-line buttons are mixed!

```
nav a {  
  color: #d1eba0;;  
  background-color: #568ac5;  
  display: flex;  
  height: 30px;  
  width: 200px;  
  text-decoration: none;  
  align-items: center;  
  justify-content: center;  
  border-radius: 0px 50px 0px 50px;  
  margin-right: 5px;  
}
```



Figure 35: With Flex technology, **button content** - for example in a menu - can be very easily aligned horizontally and vertically centered. However, two-line button labels mixed with single-line labels still prove to be difficult.

Reflection questions

As always, please edit these questions in a separate text file (e.g. MS Word) save it in the module folder with your surname and the module number as the file name!

Please save programming tasks in a separate subdirectory. For submissions via Moodle, please compress the files into a zip archive (no other compression methods!).

- 14.1 What are the two axes on which the flex items are aligned?
- 14.2 Explain the terms "parent element", "child element" and "descendant" in context with an HTML structure!
- 14.3 Which CSS rule (flex layout) do you have to choose to arrange elements horizontally from right to left?
- 14.4 Explain the difference between the CSS rules "*flex-direction: row-reverse;*" and "*flex-wrap: wrap-reverse;*"! If necessary, go back to the main topic "The flex container" from page 5.
- 14.5 You have become familiar with the "order" property. In which part of a Flex layout do you use it (Flex container, Flex item, Flex grow)?
- 14.6 What happens if you format elements with "order" and two elements have the same number a value? Also think about the order in the HTML code!
- 14.7 At which position will you find an element in a flex layout on the page if this (unlike all other elements in the container) was not formatted with "order"?
- 14.8 Program a short HTML web page with a <section> element, which you define as a flex container. This element should contain five <div> containers, each with a paragraph, whose content should be a simple numbering (1, 2, ...). Format these <div> containers with different background colors and set the flex base to 150px. The flex items should each be assigned individual values for "*flex-grow*": DIV1: 2, DIV2: 4, DIV3: 3, DIV4: 1, DIV5: 1.5 (No comma). Calculate the "x-factor" for a flex container width of 1300px. Create a table as described on page 19 and compare the widths of the respective DIV containers in the web browser.
- 14.9 Use the HTML and CSS file from task 14.8: At which flex container width are **all** elements as wide as their flex base.
- 14.10 Repeat the task from 14.8. Use the same values as before for "*flex-grow*", but this time for the "*flex-shrink*" property. Calculate the factor for a viewport width of 750px and enter the values in a table.
- 14.11 Program a simple website with a structure similar to described from page 26. However, add an additional flex item (<aside>), which should have a share of 15% in the width of the flex container. Reduce the percentage width of the <main> element shown in the example so that the <nav>, <main> and <aside> should into the width of the page without wrapping. The <header> should be in position 1 and the <footer> can now be placed at position 5.
- 14.12 Program an additional CSS code for the web page from task 14.11 with a maximum viewport width 850 screen pixels. In this case, **all** elements should be displayed across the full width. The menu should appear horizontally. This time, please use flex technology for the horizontal as well as vertically and horizontally centered display of the buttons. Make sure that the elements appear in the following order up to a maximum of 850 screen pixels: <header>, <nav>, <aside>, <main> and <footer>